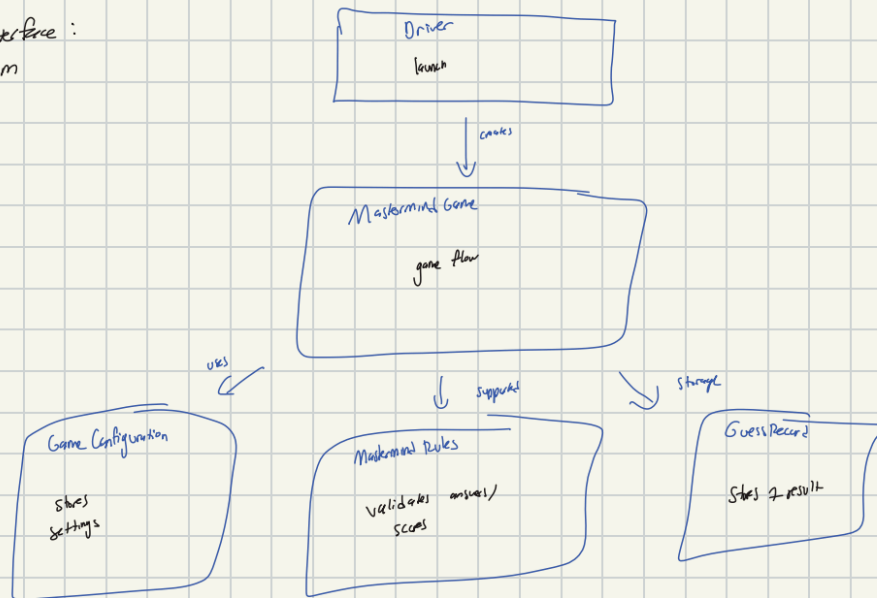


Design: 5 main classes, we separate startup, game flow, settings, rules, & results. keeps a simple implementation while allowing pt 2 to reuse.

Class/Interface:
Diagram



Responsibilities:

Driver: Checks format in terminal / launches Mastermind
 Game Configuration: stores/validates guesses, color, and code length.
 Mastermind Game: tracks history, run actual rounds, supports replay and goes through console commands.
 Mastermind Rules: calculates black/white peg feedback.
 Guess Record: stores one valid guess with black/white counter. formats history outputs also.

Reusable Pieces:	Classification	Reason
Driver	Reusable	can select/launch game from same entry point
Game Configuration	mostly reusable	settings centralized, can use similar values for game.
Mastermind Game	Partly reusable	replay flow reusable but prompts are all Mastermind.
Mastermind Rules	not really - specific to Mastermind	Black/white feedback only for this game.
Guess Record	partly reusable	history concept general, black/white specific to game.

Configuration:

We know the default configuration is 4 pegs, 12 valid guesses and BGOPRY.

GameConfiguration actually checks the constructor arguments and exposes all read-only getters. For different game sizes Driver can create a new configuration without impacting game flow.

Example: `GameConfiguration changed = newGameConfiguration(5, 8, "RGB");`